

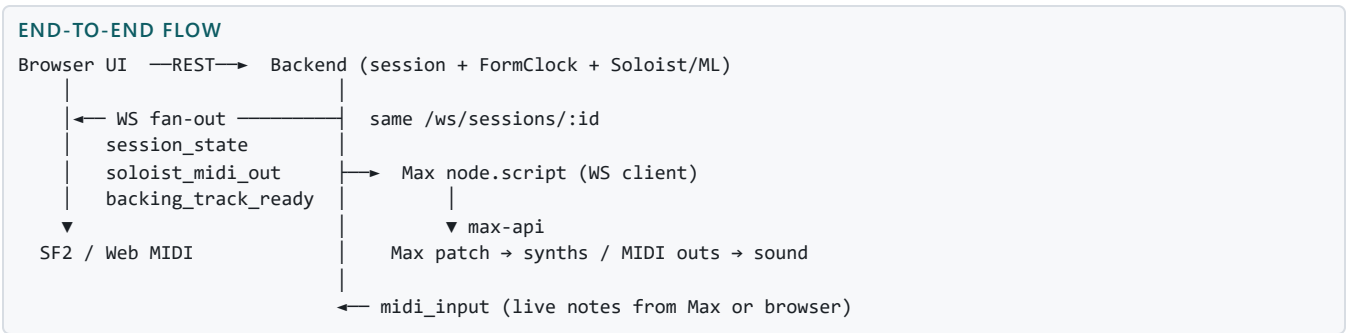
Pipeline, Dataflow, and Feature Possibilities

How model and session output reaches Max as JSON MIDI events, becomes real sound, and what that unlocks beside the web jam app.

Two-page brief · July 2026 · Bidirectional peer on the existing session WebSocket

1. Pipeline and dataflow

Core idea. Jazz Gen owns the jam (clock, turns, generation). Output leaves the backend as JSON WebSocket messages of beat-timed MIDI notes. Max (and/or the browser) turns those notes into sound. The web UI still shows session state; Max is an extra performance and audio surface, not a replacement for the app’s display unless you choose that.



What moves on the wire

MESSAGE	DIRECTION	PAYLOAD
soloist_midi_out	App → Max / browser	Final model or procedural line: list of notes (pitch, velocity, startBeat, durationBeats)
backing_track_ready	App → Max / browser	Bass, drums, comp as MIDI parts (+ optional band energy)
session_state	App → Max / browser	BPM, phase, chorus, form progress (shared clock)
midi_input	Max / browser → App	Live player notes into the session buffer (during player turns)

Step by step

1. User starts a jam in the browser. Backend creates a session and runs the form clock.
2. Max’s Node for Max script connects as a second client on `/ws/sessions/:id`.
3. At a soloist turn, SoloistService (or a future ML client) produces notes. The backend broadcasts `soloist_midi_out` as JSON.
4. Max unpacks the note list, schedules from beats + BPM, and plays real audio (or drives external MIDI gear).
5. The browser receives the same JSON and can preview with SF2 while showing the jam UI.
6. Live playing from Max or the browser returns as `midi_input` for trading and analysis.

Important: this is not PCM or WAV over the socket. The “final model output” is structured MIDI event JSON. Sound is rendered where you choose (Max, browser, or both). Generation today is usually a chorus-sized batch, so Max should schedule the phrase ahead, not treat each packet as a live note-on stream.

2. Feature possibilities

Once the pipe exists, Max can sit beside Jazz Gen in several roles. The web app stays the session brain; Max extends how the jam is heard, controlled, and staged.

Custom soloist voice

Play `soloist_midi_out` through Max synths, samples, or hardware while the browser keeps a simple preview.

Hardware / studio input

Keyboards, wind MIDI, and pedals in Max send `midi_input` into the same player buffer the trade engine already uses.

Max as house band

Use backing MIDI parts as a score. Replace or upgrade browser SF2 with Max bass, drums, and comp; map energy to dynamics.

Show control and visuals

Drive lights, video, or clip cues from phase, chorus, and BPM in `session_state` without Max needing jazz theory.

Teaching lab

Instructor Max patch mirrors turns and freezes phrases for slow playback while students jam in the browser.

External / ML generator

Later, Max or an ML host can produce the note list. Jazz Gen still clocks the session and fans JSON to the browser.

DAW and multi-out routing

Max sits between Jazz Gen and Ableton or outboard gear: separate soloist vs band channels, record stems, share clock.

Dual surface jam

One shared beat: web stage for setup and display, Max for performance audio and controllers.

What stays true across features

- Backend form clock remains master; Max follows `session_state`.
- Currency is always beat-timed MIDI JSON, not audio blobs.
- Same session id, same fan-out: browser and Max hear one jam.

Jazz Gen · Max pipeline brief · Derived from the session WebSocket bus (`/ws/sessions/:id`) and Node for Max as the Max-side client.